UNIVERSIDAD DE VALPARAÍSO
Escuela de Ingeniería Civil Informática
CIN 421 / INF423 — Prueba de Sistemas · 2° Semestre 2026

Plan de Pruebas

Proyecto TicketU · Entrega 1 · Microservicio de Pagos · Grupo Pagos

0. Identificación del plan

Campo	Detalle
Grupo QA	Grupo Pagos
Microservicio de TicketU	Pagos: creación del cobro, estado, reembolsos y consulta de pagos
Equipo TITEC del servicio	Grupo Pagos, el mismo grupo (repo Modulos-Pagos/arquitectura-pagos)
Avance / build probado	Avance 1 (01/10/2026) · Swagger 2.3.0 · front 0.1.0
Versión del plan	1.2
Fecha	23/10/2026
Responsable (rol de gestión de pruebas)	Ricardo Muñoz (gestión de pruebas)
Integrantes	Ricardo Muñoz (encargado), Simón Canales, Cristóbal Soto, Pablo Medina, Alonso Venegas y Diego Fernández

1. Introducción

1.1 Propósito y objetivos de las pruebas

- Verificar que el servicio cobre exactamente el total de la reserva y nunca dos veces la misma reserva (R1, R2).
- Verificar que el estado del pago avance solo por transiciones válidas y llegue a Entradas: PENDIENTE → APROBADO, RECHAZADO o ANULADO, y APROBADO → REEMBOLSADO (R3, R4, R5, R8).
- Verificar que cada rol acceda solo a lo que le corresponde: comprador a sus pagos, SERVICIO a todos, ADMIN a la eliminación (R6, R13).
- Dejar diseñados en TestLink 38 casos individuales (con técnica, tipo e importancia declarados) y 8 casos E2E, enlazados al 100 % de los requisitos del equipo.
- Revisar estáticamente el contrato OpenAPI, los contratos de interfaz y las historias de usuario antes de ejecutar (sección 5.4).

1.2 Base de prueba

Documento	Ubicación o enlace	Versión / fecha
Contrato OpenAPI del servicio de Pagos	Repo Modulos-Pagos/arquitectura-pagos · backend/docs/openapi.yaml (/api-docs)	v2.3.0 · 30/09/2026
Historias de usuario (HU 1.1 a 1.6, 3.1, 3.3 y 3.4)	Issues del repo Modulos-Pagos/arquitectura-pagos (Sprint 1 y 2 · E1)	28/09/2026
Contrato de interfaz Pagos ↔ Entradas/Inventario; consumo	Nuestro repo de TITEC, Modulos-Pagos/arquitectura-pagos	v2.1 · 28/09/2026

Documento	Ubicación o enlace	Versión / fecha
de Promociones en la sección «Promociones (consumidor)» del Swagger	(docs/contratos/Contrato_Pagos_Entradas_v2.1.docx)	
Especificación TITEC Evaluación 1 y tabla de dependencias	Material del curso TITEC 2026-2	2026-2
Diagrama relacional y diccionario de datos	backend/docs/base-de-datos/	30/09/2026
Enunciado «Probamos TicketU» · Entrega 1 (niveles, tipos, técnicas, entregables y pauta)	Material del curso CIN421	21/10/2026

1.3 Alcance

Dentro del alcance	Fuera del alcance	Quién lo cubre / cuándo
Crear el cobro de una reserva (POST /transacciones), idempotencia y validación del monto	Funcionamiento interno de Stripe	Stripe (tercero). Solo se prueba nuestra integración en modo test.
Estado del pago: GET /{id_pago}, webhook de Stripe y polling	Servicio real de Entradas (hoy simulado)	Grupo QA de Entradas; la integración real se prueba en la Entrega 2 o 3 (R15).
Reembolso y anulación (POST /{id_pago}/reembolso)	Emisión de tokens JWT	Grupo QA de Auth. Aquí se usan tokens generados con el secreto compartido.
Listado para Promociones (GET /pagos) y eliminación (DELETE /{id_pago})	Rendimiento y carga	Este grupo, en la Entrega 3 con Locust (R14).
Autorización por rol en todos los endpoints y front de checkout (formulario, errores y comprobante)	Pruebas de aceptación (UAT)	Profesores y ayudante de TITEC en la Entrega 3.
Revisión estática del contrato OpenAPI, los contratos de interfaz y las HU; diseño de los E2E	Ejecución de los E2E en la Entrega 1	Este grupo, en la Entrega 3, con los servicios integrados.

1.4 Supuestos y restricciones

- El build del Avance 1 corre en local: backend en http://localhost:3000 y front en http://localhost:5173, con base de datos en Supabase. Se usan dos configuraciones: Ambiente A (PASARELA=mock, para la API) y Ambiente B (Stripe test, para el front y la firma del webhook).
- Stripe se usa en modo test, con tarjetas de prueba; no se mueve dinero real.
- Entradas está simulado según su contrato (ENTRADAS_MODO=mock): no se valida contra el servicio real en esta entrega.
- Independencia: el grupo QA y el equipo que desarrolla Pagos en TITEC comparten 5 de sus 6 integrantes, así que la independencia es baja (ISTQB 5.1); el único integrante externo al desarrollo es Diego Fernández. Se compensa diseñando los casos desde el contrato y las HU (caja negra, no desde el código), con revisión cruzada de los casos entre integrantes, con la revisión estática y los E2E a cargo de Diego, y con la UAT a cargo del profesor y el ayudante de TITEC.
- Los desarrolladores del grupo (Cristóbal Soto en backend, Pablo Medina en front y Alonso Venegas en base de datos) corrigen los defectos críticos en un máximo de dos días hábiles.

- Restricción: el plan gratuito de Supabase se pausa por inactividad y Stripe test puede superar el timeout de 1.200 ms (ver R16).
- Restricción: el grupo dispone de unas 34 horas para la Entrega 1 (sección 4.5).

1.5 Partes interesadas y comunicación

Parte interesada	Rol o interés	Cómo y cuándo se comunica
Nosotros como equipo de desarrollo de Pagos en TITEC (Cristóbal Soto, Pablo Medina, Alonso Venegas)	Desarrollamos el servicio y corregimos los defectos; 5 de los 6 integrantes del grupo QA son también parte del desarrollo	Mantis (cada defecto) y revisión de defectos en la daily o el sprint de TITEC
Profesores y ayudante de CIN421	Evalúan el plan, la matriz y la ejecución	Entregas en /qa y presentación: 23/10, 06/11 y 27/11
Profesor y ayudante de TITEC	Dueños del ambiente y de la UAT	Correo y presentación de cada avance
Equipos Entradas y Promociones	Consumen los servicios de Pagos según contrato	Issues de integración en GitHub ante cambios de contrato

2. Enfoque de prueba (estrategia)

2.1 Enfoque elegido

Enfoque	¿Se usa? (sí/no)	Justificación
Analítico (basado en riesgos)	Sí	Los riesgos de la matriz definen qué se prueba primero y con qué profundidad.
Basado en modelos	No (E1) · Sí (E3)	En la Entrega 1 no se usan perfiles operativos ni modelos de fiabilidad. En la Entrega 3 la carga con Locust sigue un perfil de uso esperado (creación de cobros y polling de Entradas). La máquina de estados del pago se cubre con tablas de decisión estado × evento, dentro del enfoque analítico.
Metódico (checklists, predicción de errores)	Sí	Checklist de errores típicos de API (reintentos, firma inválida, JSON inválido, token ausente, identificadores mal formados) como fuente de clases inválidas; checklist de revisión del contrato OpenAPI.
De proceso / normas	Sí	El plan sigue IEEE 829 / ISTQB CTFL 5.2; los casos y defectos siguen las guías del curso (TestLink, Mantis).
Dinámico y heurístico (exploratorio)	No (E1)	En la Entrega 1 la casuística usa solo las técnicas pedidas; las sesiones exploratorias del front quedan para la Entrega 3.
Consultivo	Sí	El profesor y el ayudante de TITEC, externos al grupo, fijan los guiones y criterios de la UAT; el contrato y el SLA los acordamos, como equipo Pagos de TITEC, con Entradas y Promociones.
Anti-regresión	Sí	La colección Postman se re-ejecuta con Newman en cada avance.

2.2 Niveles, tipos, técnicas y herramientas por entrega

	Entrega 1 · 23/10	Entrega 2 · 06/11	Entrega 3 · 27/11
Nivel	Componente (API REST de Pagos, Entradas simulado), Sistema (diseño de los E2E desde los requisitos) y Estático (revisión de HU y del contrato OpenAPI)	Integración de API y contratos (Pagos con Entradas y Promociones, front con back). Detalle por confirmar: la entrega se está reformulando	Sistema (ejecución de los E2E), no funcionales y aceptación (UAT)
Tipos de prueba	Funcional, negativa (entradas inválidas → 4xx claros), positiva (todos los métodos funcionan) y revisión de contrato	Integración, contrato, confirmación y regresión	E2E, rendimiento y carga, regresión y UAT
Técnicas	Individual: partición de equivalencia, valores límite y tabla de decisión. E2E: casos de uso y escenarios	Las anteriores más pruebas de contrato	Casos de uso y escenarios E2E; perfiles de carga
Herramientas	TestLink (requisitos, casos y trazabilidad), Word (plan), Excel (matriz de riesgos) y Postman	TestLink, Mantis, Postman y Newman	TestLink, Mantis, Newman y Locust
Sobre qué avance	Avance 1 (01/10)	Avance 2 (22/10)	Avance 3 (05/11)

2.3 Funcionalidad a probar

Característica	Descripción	Riesgo asociado	Profundidad de prueba
Crear el cobro	POST /transacciones: valida el pedido, verifica el total con Entradas e inicia el cobro en Stripe	R1, R2, R7 (alto)	A fondo: varias técnicas, positivos y negativos, retest y E2E
Estado y confirmación	GET /{id_pago}, webhook de Stripe y polling de respaldo	R3, R4, R8 (alto)	A fondo: todas las transiciones válidas e inválidas
Reembolso y anulación	POST /{id_pago}/reembolso para pagos PENDIENTES o APROBADOS	R5, R4 (alto)	A fondo, con reintentos
Autorización	JWT de Auth y política por rol en cada endpoint	R6 (alto)	A fondo: tabla de decisión completa
Listado y eliminación	GET /pagos para Promociones y DELETE /{id_pago} solo para ADMIN	R12, R13 (medio)	Técnica principal y valores límite
Front de checkout y pasarela	Formulario con Stripe Elements, tarjeta rechazada, pasarela caída y comprobante	R9, R10 (medio) · R11 (bajo)	Partición de equivalencia; comprobante por estado del pago
Flujos E2E de TicketU	Compra pagada, evento gratuito, reembolso, historial y Promociones (cruzan servicios)	R1–R6, R10–R12	Casos de uso y escenarios: se diseñan ahora y se ejecutan en la Entrega 3

2.4 Técnicas de diseño

Técnica	Dónde se aplica	Riesgos que cubre
Partición de equivalencia	Body de POST /transacciones, identificadores y filtros; firma y eventos del webhook; tarjetas de prueba; campos del formulario; estado del comprobante	R1, R2, R4, R7, R8, R9, R10, R11, R12
Valores límite	total (-1, 0, 1, 99.999.999, 100.000.000), cantidad_entradas (0, 1, 100, 101), motivo (2, 3, 255, 256), limite (0, 1, 100, 101) y pagina (0, 1)	R2, R7, R12
Tabla de decisión	Idempotencia (reserva × total × usuario); estado del pago × evento de Stripe; estado × reembolso; rol × operación × dueño; estado × rol en DELETE	R1, R3, R4, R5, R6, R13
Casos de uso y escenarios (E2E)	Flujos de TicketU derivados de los requisitos RS-01 a RS-04 (básico, alternativos y de excepción)	R1–R6, R10–R12
Revisión estática (checklist)	Contrato OpenAPI, contratos de interfaz e HU: completitud, consistencia y coherencia con la arquitectura de TicketU	R14, R17

2.5 Criterios de entrada, salida, suspensión y reanudación

Criterio	Condición medible	Cómo se verifica
Entrada	El build del avance responde, el contrato OpenAPI está publicado en /api-docs, los datos de prueba están cargados (tokens por rol y reservas qa-*) y los casos de riesgo alto están diseñados y enlazados a requisitos	TC-PAG-38: GET /health responde 200; los 15 requisitos tienen casos según la matriz de trazabilidad (sección 5.3)
Salida	100 % de los casos de riesgo alto ejecutados, ≥ 90 % de los de riesgo medio, 0 defectos de severidad bloqueo o crítico abiertos, y todo defecto mayor corregido con su retest ejecutado	Reporte de ejecución de TestLink y filtro de Mantis por severidad y estado
Suspensión	El ambiente no está disponible por más de 2 horas, o un defecto bloquea ≥ 30 % de los casos	Defecto con severidad bloqueo en Mantis y casos marcados como bloqueados en TestLink
Reanudación	El defecto bloqueante está resuelto y su caso de confirmación pasa, y GET /health responde 200. Al reanudar se re-ejecutan primero los casos bloqueados	Estado "resuelto" en Mantis y ejecución exitosa del caso de confirmación en TestLink

2.6 Retest y regresión

Confirmación: cada defecto corregido se re-ejecuta con su caso en el build que lo corrige y solo se cierra en Mantis si pasa. Regresión: en cada nuevo avance se ejecuta con Newman la colección Postman completa (humo + riesgos altos) antes de probar lo nuevo; los casos de riesgo alto se repiten además en TestLink.

2.7 Rendimiento y carga

Foco: POST /transacciones (SLA < 2.000 ms), GET /{id_pago} durante el polling de Entradas (< 2.000 ms si debe confirmar con Stripe; < 300 ms con estado final; propuesta del contrato v2.1) y GET /pagos (< 300 ms). Se prueba

en la Entrega 3 con Locust, con la pasarela simulada (PASARELA=mock) para no saturar Stripe. El detalle va en el plan de carga.

2.8 UAT

La ejecutan el profesor o el ayudante de TITEC como comprador, en la Entrega 3, con tres guiones: compra pagada aprobada, tarjeta rechazada y reintento, y reembolso por reserva vencida. El grupo QA prepara los guiones, los datos y los tokens, y agenda la sesión con el ayudante la semana del 23/11.

2.9 Gestión de defectos

Proyecto de Mantis "TicketU – Pagos". Severidad: bloqueo (impide probar), crítico (dinero mal cobrado, cobro duplicado o acceso a pagos ajenos), mayor (falla una función sin alternativa), menor y trivial. La prioridad sigue el nivel de riesgo del caso. Asignación: backend a Cristóbal Soto, front a Pablo Medina y base de datos a Alonso Venegas, en su rol de desarrolladores del grupo en TITEC; el gestor de pruebas hace el triage.

3. Riesgos

ID	Riesgo	Tipo	P	I	Nivel	Respuesta	Técnica o acción
R1	Se cobra dos veces la misma reserva	Producto	2	3	6 · Alto	Mitigar con pruebas	Tabla de decisión de idempotencia (reserva × total × usuario) y solicitudes simultáneas
R2	Se cobra un monto distinto al total de la reserva	Producto	2	3	6 · Alto	Mitigar con pruebas	Valores límite sobre total y partición de equivalencia contra la reserva de Entradas
R3	El pago no cambia a APROBADO tras pagar y Entradas no confirma las entradas	Producto	3	3	9 · Alto	Mitigar con pruebas	Tabla de decisión estado × evento: webhook, consulta sin webhook y sesión expirada
R4	Un pago cambia a un estado no permitido (por ejemplo, APROBADO a RECHAZADO)	Producto	2	3	6 · Alto	Mitigar con pruebas	Tabla de decisión estado × evento: eventos tardíos sobre estados finales; tarjeta rechazada
R5	El reembolso se hace dos veces o sobre un pago que no correspondía	Producto	2	3	6 · Alto	Mitigar con pruebas	Tabla de decisión estado × reembolso y partición del reintento idempotente
R6	Un comprador ve o reembolsa pagos de otro usuario	Producto	2	3	6 · Alto	Mitigar con pruebas	Tabla de decisión: rol × operación × dueño del pago
R7	Una solicitud inválida se acepta o el error no indica qué corregir	Producto	2	2	4 · Medio	Mitigar con pruebas	Partición de equivalencia y valores límite sobre el body, los parámetros y el motivo
R8	Se acepta un aviso falso de Stripe o un aviso repetido se procesa dos veces	Producto	2	3	6 · Alto	Mitigar con pruebas	Partición de equivalencia: firma inválida, sin firma, cuerpo inválido y evento duplicado
R9	El formulario de pago no avisa errores de la tarjeta	Producto	2	2	4 · Medio	Mitigar con pruebas	Partición de equivalencia de los campos del

ID	Riesgo	Tipo	P	I	Nivel	Respuesta	Técnica o acción
	o deja enviar datos incompletos						formulario (vacíos e inválidos)
R10	Ante tarjeta rechazada o pasarela caída el usuario no recibe un mensaje claro	Producto	2	2	4 · Medio	Mitigar con pruebas	Partición de equivalencia: tarjeta rechazada, sin fondos y pasarela sin respuesta
R11	El comprobante muestra datos incorrectos	Producto	1	2	2 · Bajo	Aceptar	Pocos casos: partición por estado del pago en la pantalla de comprobante
R12	El listado para Promociones filtra o pagina mal	Producto	2	2	4 · Medio	Mitigar con pruebas	Valores límite (pagina, limite) y partición de equivalencia de filtros
R13	Se elimina un pago que movió dinero o lo elimina un rol no autorizado	Producto	1	3	3 · Medio	Mitigar con pruebas	Tabla de decisión: estado del pago × rol
R14	Los tiempos de respuesta superan el SLA del contrato con carga	Producto	2	2	4 · Medio	Transferir	A la Entrega 3: carga con Locust (plan de carga aparte)
R15	El servicio real de Entradas no está disponible y no se puede probar la integración	Proyecto	3	2	6 · Alto	Plan de contingencia	Probar contra el mock según el contrato; la integración real pasa a la Entrega 2 o 3
R16	El ambiente de prueba no está disponible (Supabase pausado o Stripe lento)	Proyecto	3	2	6 · Alto	Plan de contingencia	Reanudar Supabase y correr TC-PAG-38 (health) antes de cada sesión; plan B: PostgreSQL local con docker-compose y PASARELA=mock
R17	El contrato de la API cambia entre avances y deja casos desactualizados	Proyecto	2	2	4 · Medio	Plan de contingencia	Revisar el Swagger y el contrato al inicio de cada entrega y versionar la colección
R18	Poca disponibilidad del grupo por la carga académica	Proyecto	2	2	4 · Medio	Plan de contingencia	Priorizar los riesgos altos y repartir los casos por integrante con fecha interna

Escala: probabilidad e impacto de 1 a 3; nivel = P × I. Alto 6–9 · Medio 3–4 · Bajo 1–2 (convención del curso, matriz 3 × 3). Registro completo de los 18 riesgos (14 de producto y 4 de proyecto) con causa, casos TestLink, responsable y estado en Matriz_Riesgos_Pagos_Entrega1.xlsx.

Orden de ejecución: R2 → R1 → R3 → R8 → R4 → R5 → R6 → R7 → R10 → R12 → R13 → R9 → R11 → R14. Primero se valida la creación del cobro (R2, R1), porque los demás casos parten de un pago creado; luego el estado y los avisos de Stripe (R3, R8, R4), el reembolso (R5) y la autorización (R6). R14 se ejecuta en la Entrega 3.

4. Recursos e infraestructura

4.1 Roles y recursos

Integrante	Rol	Responsabilidades	% dedicación
Ricardo Muñoz	Encargado · gestión de pruebas	Plan, matriz de riesgos, Test Plan en TestLink, triage en Mantis, reporte; riesgos de proyecto (R14, R16, R18)	20 %
Simón Canales	Testing (API)	Casos de creación del cobro, validación y listado (R1, R2, R7, R12)	16 %
Cristóbal Soto	Testing (API)	Casos de estados, webhook, reembolso y eliminación (R3, R4, R5, R8, R13)	16 %
Pablo Medina	Testing (front)	Front de checkout: formulario, tarjeta rechazada y comprobante (R9, R10, R11)	16 %
Alonso Venegas	Testing (seguridad)	Autorización por rol en todos los endpoints (R6) y datos de prueba	16 %
Diego Fernández	Testing (sistema) y revisión estática	Casos E2E, requisitos y trazabilidad en TestLink, revisión del contrato OpenAPI (R15, R17)	16 %

4.2 Ambientes de prueba

Backend en http://localhost:3000 (Swagger en /api-docs) y front en http://localhost:5173, sobre la rama main del repo Modulos-Pagos/arquitectura-pagos (Avance 1, Swagger 2.3.0), con base de datos en Supabase. Ambiente A (API): PASARELA=mock y ENTRADAS_MODO=mock; los pagos se aprueban simulando el webhook, como en la colección Postman. Ambiente B (front y webhook firmado): PASARELA=stripe en modo test, Stripe CLI con stripe listen y NEXT_PUBLIC_PAGOS_TOKEN del comprador usr-qa-1. TC-PAG-05 necesita el servicio real de Entradas y queda bloqueado en el Avance 1. Plan B: PostgreSQL local con docker-compose y PASARELA=mock; si el build no levanta, se reporta como bloqueo en Mantis y se prueba con el build anterior.

4.3 Datos de prueba

Tokens JWT por rol generados con npm run token: comprador usr-qa-1, otro comprador usr-qa-2, SERVICIO svc-entradas y ADMIN usr-admin. Cada caso usa un id_reserva propio con prefijo qa-<caso>-<n>. Tarjetas de prueba de Stripe: 4242 4242 4242 4242 (aprobada), 4000 0000 0000 0002 (rechazada) y 4000 0000 0000 9995 (fondos insuficientes). Sin datos personales reales: el comprador de ejemplo es ficticio. Al cerrar cada sesión, los pagos anulados o rechazados se eliminan con DELETE (ADMIN).

4.4 Herramientas

Herramienta	Uso en este plan
TestLink	Proyecto compartido «TichetU – Entrega 1», carpeta 4 · Pagos: 46 casos (38 en Casuística individual, en 6 suites, y 8 en E2E) con técnica, tipo, nivel y escenario en los campos personalizados del curso. La importación de los 15 requisitos (XML listo) está pendiente de que se habilite el módulo de requisitos en el proyecto (solicitado a la docente). Mientras tanto, la trazabilidad se declara en el resumen de cada caso y en la hoja «Trazabilidad» del Excel.
Mantis	Proyecto "TicketU – Pagos": reporte y seguimiento de defectos con severidad y prioridad
Postman / Newman	Nuestra colección Pagos-TITEC (hecha en TITEC) como base, más los casos TC-PAG; Newman para la regresión

Herramienta	Uso en este plan
Locust	Entrega 3: carga sobre POST /transacciones y GET /{id_pago}
Repositorio /qa	Plan, matriz de riesgos, colecciones, entornos sin secretos y reportes de ejecución

4.5 Estimación del esfuerzo

Actividad	Esfuerzo (horas)	Base de la estimación	Responsable
Análisis y diseño de casos	14	Juicio de expertos: 46 casos × ~15 min de diseño, más requisitos, matriz, plan y revisión estática	Ricardo Muñoz y Diego Fernández
Implementación (datos, colecciones)	8	Juicio de expertos: se parte de nuestra colección Postman de TITEC (28 requests)	Simón Canales y Cristóbal Soto
Ejecución y registro	6	Juicio de expertos: ~10 min por caso, priorizando los de riesgo alto	Testers
Reporte de defectos y retest	6	Supuesto: ~6 defectos × 1 h (reporte, retest y cierre)	Ricardo Muñoz y testers

4.6 Calendario

Hito	Fecha	Sobre qué avance
Entrega 1 y presentación (15 min)	23/10	Avance 1 (01/10)
Entrega 2 y presentación (20 min)	06/11	Avance 2 (22/10)
Entrega 3, UAT y presentación (20 min)	27/11	Avance 3 (05/11)
Hito interno: importar los 15 requisitos (XML listo), enlazarlos a los 46 casos y armar el Test Plan en TestLink con el plan y la matriz (Ricardo Muñoz)	Al habilitarse el módulo de requisitos (solicitado a la docente)	Avance 1 (01/10)
Hito interno: ejecución de los casos individuales (primero riesgo alto) y reporte en Mantis (testers)	26/10 – 30/10	Avance 1 y luego Avance 2 (22/10)
Hito interno: retest de defectos corregidos y regresión con Newman (testers)	02/11 – 04/11	Avance 2 (22/10)
Hito interno: ejecución de los E2E y carga con Locust (Diego Fernández y testers)	09/11 – 20/11	Avance 3 (05/11)
Hito interno: sesión UAT con el ayudante de TITEC (Ricardo Muñoz)	Semana del 23/11	Avance 3 (05/11)

5. Casuística y trazabilidad

5.1 Casuística individual

Técnica	Riesgo alto	Riesgo medio	Riesgo bajo
Partición de equivalencia	9	8	1
Valores límite	2	3	0
Tabla de decisión	12	3	0
Total	23	14	1

Por tipo: 13 positivos, 14 negativos y 11 que combinan ambos. Todos los métodos tienen al menos un caso positivo: POST /transacciones (TC-PAG-01), GET /{id_pago} (TC-PAG-08), GET /{id_pago}/checkout (TC-PAG-36), POST /{id_pago}/reembolso (TC-PAG-13), GET /pagos (TC-PAG-19), DELETE /{id_pago} (TC-PAG-33), webhook (TC-PAG-08) y GET /health (TC-PAG-38). Detalle de cada caso: XML y Excel de TestLink.

5.2 Casuística E2E

Flujo	Pasos y servicios involucrados	Riesgos	Casos TestLink
Compra de entradas para un evento pagado (flujo básico)	Auth → Catálogo → Entradas → Pagos → Stripe → Entradas → Notificaciones. Catálogo → Entradas reserva → Pagos crea el cobro → paga con 4242… → APROBADO → Entradas emite 2 QR → Notificaciones avisa una vez	R1, R2, R3	E2E-PAG-01 (RS-01, HU 1.1, HU 1.3, HU 1.6, HU 1.5)
Tarjeta rechazada y reintento con otra tarjeta	Entradas → Pagos → Stripe → Entradas. Paga con 4000…0002 → rechazo, sigue PENDIENTE, sin entrada → reintenta con 4242… → APROBADO; un solo cobro	R4, R10, R1	E2E-PAG-02 (RS-01, HU 1.4, HU 1.6)
Doble clic en «Comprar» o reintento de Entradas: un solo cobro	Entradas → Pagos. Doble clic o reintento de Entradas → mismo id_pago → un cobro y una emisión	R1	E2E-PAG-03 (RS-01, HU 1.1)
La reserva vence antes de pagar: se anula sin cobrar	Entradas → Pagos. No paga → vence la reserva → Entradas pide anular → ANULADO; el stock vuelve	R5	E2E-PAG-04 (RS-03, HU 3.3)
Pago aprobado sin stock o con reserva vencida: reembolso automático	Entradas → Pagos → Stripe → Notificaciones. Dos compradores por la última entrada → uno sin stock → Entradas pide reembolso → REEMBOLSADO y aviso	R5	E2E-PAG-05 (RS-03, HU 3.1, HU 3.3)
Reservo una charla gratuita y entro con mi QR (sin pasar por Pagos)	Auth → Catálogo → Entradas → Check-in. Reserva charla gratuita → QR sin pantalla de pago → GET /pagos?id_reserva= devuelve [] → Check-in valida el QR	R2	E2E-PAG-06 (RS-02, HU 1.1)
El comprador revisa su historial y su comprobante	Auth → Pagos. Mi cuenta → historial solo con sus pagos → comprobante coincide con GET /{id_pago}; otro usuario recibe 403	R6, R11	E2E-PAG-07 (RS-01, HU 3.4, HU 1.5)
Promociones consulta las compras	Promociones → Pagos. Promociones consulta GET /pagos?id_usuario=&estado=APROBADO	R12	E2E-PAG-08 (RS-04, HU 3.4)

Flujo	Pasos y servicios involucrados	Riesgos	Casos TestLink
pagadas de un usuario	→ beneficio solo por pagos aprobados		

5.3 Matriz de trazabilidad

Requisito / HU	Riesgo	Casos TestLink	Importancia	Estado
HU 1.1 Redirección a checkout seguro (crear el cobro)	R1, R2, R3, R6, R7	TC-PAG-01, 02, 03, 04, 05, 06, 07, 16, 22, 36; E2E-PAG-01, E2E-PAG-03, E2E-PAG-06	Alta	Diseñado; TC-PAG-05 bloqueado
HU 1.2 Validación de la tarjeta en el formulario	R9	TC-PAG-25, 26	Media	Diseñado
HU 1.3 Integración con la pasarela externa (webhook de Stripe)	R1, R2, R3, R4, R8	TC-PAG-08, 10, 12, 23, 24; E2E-PAG-01	Alta	Diseñado
HU 1.4 Manejo de rechazos y errores	R1, R2, R3, R4, R7, R10	TC-PAG-06, 07, 10, 11, 20, 21, 22, 27, 28; E2E-PAG-02	Alta	Diseñado
HU 1.5 Comprobante de la transacción	R1, R2, R3, R6, R11	TC-PAG-29; E2E-PAG-01, E2E-PAG-07	Alta	Diseñado
HU 1.6 Confirmación del pago a Entradas (consulta de estado)	R1, R2, R3, R4, R6, R10	TC-PAG-08, 09, 16, 17, 19; E2E-PAG-01, E2E-PAG-02	Alta	Diseñado
HU 3.1 Reembolso total	R5, R6, R7	TC-PAG-13, 14, 15, 18, 19, 37; E2E-PAG-05	Alta	Diseñado
HU 3.3 Reversión por reserva vencida o falta de stock	R5	TC-PAG-13, 14; E2E-PAG-04, E2E-PAG-05	Alta	Diseñado
HU 3.4 Historial de pagos y listado para Promociones	R6, R11, R12	TC-PAG-16, 18, 19, 30, 31, 32; E2E-PAG-07, E2E-PAG-08	Alta	Diseñado
Operaciones sobre la BD: crear, consultar, modificar y eliminar (requisito TITEC)	R13	TC-PAG-33, 34, 35	Media	Diseñado
Servicio operativo y contrato OpenAPI publicado (requisito TITEC)	R16	TC-PAG-38	Alta	Diseñado
RS-01 Compra de entradas para un evento pagado	R1, R2, R3, R4, R6, R10, R11	E2E-PAG-01, E2E-PAG-02, E2E-PAG-03, E2E-PAG-07	Alta	Diseñado (E2E: ejecución en E3)
RS-02 Reserva de entradas para un evento gratuito	R2	E2E-PAG-06	Alta	Diseñado (E2E: ejecución en E3)
RS-03 Anulación o devolución cuando la	R5	E2E-PAG-04, E2E-PAG-05	Alta	Diseñado (E2E: ejecución en E3)

Requisito / HU	Riesgo	Casos TestLink	Importancia	Estado
reserva vence o no hay stock				
RS-04 Beneficios de Promociones según compras pagadas	R12	E2E-PAG-08	Media	Diseñado (E2E: ejecución en E3)

Cobertura: 15 de 15 requisitos tienen casos (100 %) y los 46 casos están enlazados a al menos un requisito. Cuando se habilite el módulo de requisitos, la matriz se obtendrá desde TestLink con el informe de cobertura; hoy la respalda la hoja «Trazabilidad» del Excel de casos.

5.4 Revisión estática (requisitos y contrato OpenAPI)

Checklist aplicado al Swagger 2.3.0, al contrato Pagos ↔ Entradas (v2.1), a la sección de Promociones del Swagger y a las HU: cada operación documenta sus respuestas 2xx y 4xx con el formato de error común, la política de autorización y ejemplos; los criterios de las HU coinciden con el contrato; y el diseño es coherente con la arquitectura de TicketU. Los hallazgos se registran como issues en nuestro repo de TITEC (Modulos-Pagos/arquitectura-pagos) y quedan abiertos hasta corregirlos.

ID · Artefacto	Hallazgo	Severidad	Acción propuesta	Estado
EST-01 · Arquitectura TicketU vs. Pagos	La arquitectura de TicketU indica que Entradas y Pagos se coordinan por el broker, nunca con una llamada directa. Pagos usa llamadas REST directas (Entradas consulta GET /{id_pago} y Pagos invoca GET /entradas/reservas/{id}).	Alta	Confirmar con TITEC si el MVP con polling es válido; el broker figura como evolución en el Swagger.	Abierto
EST-02 · Arquitectura TicketU vs. Pagos	TicketU define MongoDB por servicio; Pagos usa PostgreSQL (Supabase).	Baja	Informativo: confirmar que se acepta otra base de datos.	Abierto
EST-03 · Contrato v2.1 y Swagger vs. código · reembolso	El contrato (§4.4) y el Swagger dicen que el reembolso lo pide Entradas con rol SERVICIO, pero el servicio también deja reembolsar al comprador dueño del pago (TC-PAG-18). No está definido si el comprador puede pedir reembolsos.	Media	Definir en el grupo si el comprador puede reembolsar y ajustar el contrato o la política de autorización.	Abierto
EST-04 · OpenAPI · POST /transacciones	La descripción dice que crea «una sesión de Stripe Checkout», pero el método por defecto es formulario (PaymentIntent).	Media	Describir ambos métodos y el valor por defecto.	Abierto
EST-05 · Contrato Entradas ↔ Pagos	No dice qué pasa con un evento gratuito: Pagos rechaza total 0 (400). Entradas debe emitir sin llamar a Pagos.	Media	Agregar la regla al contrato y a la HU 1.1.	Abierto
EST-06 · HU 1.4 · tarjeta rechazada	Con el formulario, una tarjeta rechazada deja el pago en PENDIENTE (sin fecha de expiración). La HU solo define el caso de sesión expirada.	Media	Documentar el estado tras el rechazo y que Entradas debe anular el pago cuando vence la reserva.	Abierto

ID · Artefacto	Hallazgo	Severidad	Acción propuesta	Estado
EST-07 · OpenAPI · GET /{id_pago}	El SLA de consulta del contrato v2.1 (< 300 ms con estado final; < 2.000 ms si debe confirmar con Stripe; propuesta por confirmar) no aparece en el Swagger; sí aparece el de creación (< 2.000 ms).	Baja	Confirmar el SLA con Entradas y agregarlo a la descripción del endpoint.	Abierto
EST-08 · Backlog · HU 1.2 y HU 1.5	Estaban marcadas como backlog futuro, pero el front ya tiene formulario con Stripe Elements y pantalla de comprobante.	Baja	Definir en el grupo si están dentro del alcance del Avance 1 y actualizar el backlog.	Abierto

6. Resultados y reporte

Al cierre de cada entrega se publica en /qa/reportes: casos diseñados, ejecutados, pasados, fallidos y bloqueados por nivel de riesgo (TestLink); defectos por severidad y estado (Mantis); y riesgos altos mitigados sobre el total (matriz). Durante la entrega, el avance se revisa en la daily o el sprint de TITEC, ya que el equipo de desarrollo es el mismo grupo.

7. Control de versiones

Versión	Fecha	Cambio	Autor
1.0	20/10/2026	Versión inicial para la Entrega 1: alcance, estrategia, 18 riesgos y 35 casos planificados	Ricardo Muñoz
1.1	21/10/2026	Ajuste al enunciado de la Entrega 1: fechas 23/10, 06/11 y 27/11; nivel estático y revisión de contrato; tipos positivo y negativo; técnicas particiones, límites, tablas de decisión y casos de uso; 38 casos individuales, 8 E2E y trazabilidad a 15 requisitos	Ricardo Muñoz
1.2	23/10/2026	Revisión previa a la entrega: independencia del grupo QA (mismo equipo de TITEC); tabla de riesgos completa (18); contrato v2.1 como base de prueba; EST-03 reemplazado (el anterior era incorrecto) y SLA de consulta corregido en §2.7 y EST-07; enfoques basado en modelos y consultivo; criterios de entrada, salida y reanudación; estimación (34 h) e hitos del calendario	Grupo Pagos